Lab 3

COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Name: Sahana Shivakumar Badiger

SRN: PES2UG24CS427

Section: G

Problem Statement : Freelance Content Creator Escrow Platform

Architecture Selection:

Microservices Architecture is chosen for the Freelance Content Creator Escrow Platform System.

This platform handles distinct concerns with very different risk, scale, and compliance profiles — user identity, contract/milestone logic, real money held in escrow, file delivery, dispute handling, and notifications — which makes an architecture of small, independently deployable services a better fit than a single Layered application or a centralized Client-Server model.

Architectural Justification:

Independent Scaling: Load patterns differ sharply across components — content browsing/delivery is continuous, while Escrow Payment sees bursty spikes whenever milestones are approved. Microservices let each component scale on its own, so payment and file-delivery infrastructure can be scaled up independently of the rest of the platform without over-provisioning everything.

Fault Isolation for Financial Integrity: Since the platform holds real client funds in escrow, a failure in a non-critical service (e.g., Notification) must never affect the correctness or availability of the Escrow Payment component. Microservices give each component its own deployment and failure boundary, so an outage in messaging or

dispute handling can't corrupt or block an in-progress escrow transaction.

Security : Isolating Escrow Payment as its own service confines sensitive payment data and card-network communication to one tightly access-controlled service with its own encryption and audit logging. Other components only ever see the ContractInfo/PaymentStatus interfaces, never raw payment data — shrinking the attack surface and narrowing PCI-DSS compliance scope to a single service instead of the whole platform.

Better Performance : Because services deploy independently, Escrow Payment can run on infrastructure tuned for low-latency transaction processing, while Content Delivery & Storage can be scaled/cached separately for large file traffic. This keeps heavy file uploads from creators competing with time-sensitive payment processing for the same resources.

UML Component Diagram:

